Customer Issue Root-Cause Analysis

Project Explained in Simple Terms

The resume bullet this document explains

Resolved 30+ customer-reported issues with 50% faster mean time to resolution by leading root-cause analysis with GDB, Valgrind, Wireshark, and control plane trace logs, then delivering corrective actions and RCA reports.

Contents

1. Overall Summary (Simple Terms)
2. Key Words Explained
3. Components Explained (End to End)
4. Workflow Explained (End to End)
5. Architecture Diagram Explained (End to End)
6. A Full Example Case, Start to Finish
7. How the 50% Faster Resolution Was Achieved
8. Quick Recap and Interview Talking Points

1. Overall Summary (Simple Terms)

In one sentence: customers were having problems with a networking product (crashes, freezes, memory running out, network traffic going missing), and this work was about being the "detective" who found out **exactly why** each problem happened, fixed it, and wrote a clear report explaining it.

> **Think of it like a car mechanic**
>
> A customer brings in a car that keeps stalling. A poor mechanic just replaces random parts. A good mechanic plugs in diagnostic tools, reads the engine logs, recreates the stall on a test track, finds the one broken part that is the real cause, replaces it, and gives the customer a written note explaining what was wrong and how it was prevented from happening again.
>
> In this project, the "car" is network software (for example, software that runs on routers and switches), the "diagnostic tools" are **GDB, Valgrind, Wireshark and control plane trace logs**, and the "written note" is the **RCA (Root-Cause Analysis) report**.

What the bullet point is really saying

Part of the bullet	Plain meaning
Resolved 30+ customer-reported issues	More than 30 real problems that real customers complained about were fully fixed and closed.
50% faster mean time to resolution (MTTR)	On average, a problem now got fixed in half the time it used to take. If an issue used to take 10 days, it now took about 5.
Leading root-cause analysis	Not just applying quick patches, but driving the investigation to find the true, underlying reason for each failure.
GDB	A debugger. Used to look inside a crashed program and see the exact line of code where it died.
Valgrind	A memory checker. Used to catch memory leaks and memory misuse that slowly break a program.
Wireshark	A packet analyzer. Used to look at every message travelling over the network and spot bad or missing ones.
Control plane trace logs	Detailed diaries written by the networking software about how it decides where traffic goes. Used to rebuild a timeline of what happened.

Part of the bullet	Plain meaning
Delivering corrective actions	Shipping the real fix: a code patch, a hotfix release, or a safe configuration workaround.
RCA reports	Written documents that explain what broke, why, how it was fixed, and how it will be prevented in future.

Why this matters

- **Customers stay happy:** their networks go down for less time.
- **Problems do not come back:** fixing the root cause stops the same bug returning next month.
- **The team learns:** every RCA report becomes knowledge that makes the next investigation faster.
- **Business impact:** fewer escalations, lower support cost, and more trust in the product.

2. Key Words Explained

Before going deeper, here are the words that appear throughout this document, explained without jargon.

Term	Simple explanation
Bug / defect	A mistake in the software that makes it behave wrongly.
Root cause	The single deepest reason a problem happened. Fix it and the problem goes away for good.
Symptom	What the customer sees (for example, "the router rebooted"). Not the same as the cause.
RCA	Root-Cause Analysis: the investigation that finds the root cause, and the report describing it.
MTTR	Mean Time To Resolution: the average time from "customer reported it" to "it is fixed".
Crash	The program suddenly stops because it did something illegal (for example, read memory it did not own).
Core dump (core file)	A snapshot of the program's memory saved at the exact moment it crashed. Like a photo of the crash scene.
Stack trace / backtrace	The list of functions the program was inside when it crashed, from the newest to the oldest. Like breadcrumbs.
Memory leak	The program keeps taking memory and never gives it back, until it runs out and fails.
Use-after-free	The program keeps using a piece of memory after it already gave it back. Leads to random corruption.
Packet	A small envelope of data sent over the network.
PCAP	A file that records network packets so they can be studied later. Wireshark opens it.
Control plane	The "brain" of a network device: the part that decides routes, talks to neighbours and keeps tables up to date.
Data plane	The "muscles" of a network device: the part that actually moves packets as fast as possible.

Term	Simple explanation
Protocol (BGP, OSPF, etc.)	A set of rules routers use to talk to each other and share where networks are.
Route flapping	A route that keeps appearing and disappearing, confusing the whole network.
Reproduce (repro)	Making the same problem happen again on purpose in a lab, so it can be studied safely.
Debug symbols	Extra information inside a program that lets tools show real function names and line numbers.
Hotfix / patch	A small, quick software update that fixes a specific problem.
Workaround	A temporary way to avoid the problem (often a configuration change) until the real fix arrives.
Regression test	A test that checks the fix did not break anything that used to work.

3. Components Explained (End to End)

This section walks through every building block of the project, in the order a customer issue meets it. Each component has a simple definition, an everyday analogy, its role in this project, the fine details, and a small example.

1. The Customer Network and Product Software

What it is (simple words)

The real environment where the problem happens: the customer's routers, switches or network appliances, all running the company's software.

Everyday analogy

The patient in a hospital. Everything starts because the patient feels unwell.

What it does in this project

- It is where the failure first appears, usually under real traffic and real conditions that are hard to copy.
- It is the source of the most valuable evidence (crash files, logs, traffic captures).

Minute details

- Software is typically written in C or C++, which is fast but makes memory mistakes easy.
- It is split into processes (separate running programs), such as a routing process, a management process and a forwarding process.
- Customers run different software versions, hardware models and configurations, so the same bug may only show up for a few customers.

Small example

A large customer's core router reboots every few days during peak hours. Nobody else has reported it yet.

2. Customer-Reported Issue (Support Case)

What it is (simple words)

The formal complaint the customer opens with technical support, describing what went wrong.

Everyday analogy

The patient describing symptoms to the front desk.

What it does in this project

- Starts the clock for MTTR (mean time to resolution).
- Gives the first clues: what failed, when, how often, and how badly it hurts the business.

Minute details

- Usually contains: symptom description, software version, device model, time of failure, and business impact.
- Severity levels decide urgency. For example, Severity 1 means the network is down; Severity 3 means a minor annoyance.
- Often first-line support tries simple fixes, then escalates to engineering (where this RCA work happens).

Small example

Case: "Router R1 rebooted at 02:14 UTC. Third time this week. Production impact. Severity 2."

3. Ticketing / Bug Tracking System

What it is (simple words)

The software that stores and tracks every case and bug, such as Jira or Salesforce.

Everyday analogy

The hospital's patient file system.

What it does in this project

- Keeps one single record of everything: evidence, notes, decisions, status and owners.
- Measures time stamps (opened, assigned, root-caused, fixed, closed) used to calculate MTTR.

Minute details

- Customer support case is linked to an internal engineering bug.
- Status moves through stages: New, Triaged, Investigating, Root Cause Found, Fix in Progress, Verified, Closed.
- Search lets the engineer find older bugs with the same crash signature (duplicate detection).

4. Triage

What it is (simple words)

The quick first look that decides how serious the issue is, who owns it, and what to do first.

Everyday analogy

The nurse at the emergency room deciding who needs a doctor right now.

What it does in this project

- Prevents time being wasted on the wrong problem or the wrong team.
- Spots known bugs early. If the issue is already fixed in a newer version, the answer can be given in hours, not days.

Minute details

- Checks: severity, number of customers affected, software version, recent changes (upgrade, config change, new traffic).

- Classifies the failure type: crash, hang, memory growth, wrong forwarding, protocol instability, performance.
- The failure type decides which tool is used first: crash leads to GDB, memory growth leads to Valgrind, traffic loss leads to Wireshark, protocol instability leads to trace logs.

Small example

Router reboot plus a core file present means "crash" type, so GDB analysis is the first step.

5. Evidence Collection (the "Clues")

What it is (simple words)

Gathering every piece of data that could explain the failure.

Everyday analogy

A detective collecting fingerprints, camera footage and witness statements from the crime scene.

What it does in this project

- Good evidence is the biggest factor in fast resolution. Missing data means asking the customer again, which adds days.

Minute details

- **Core dump:** memory snapshot from the crashed process (opened with GDB).
- **Process memory data:** memory usage over time, heap statistics, or the process run under Valgrind in the lab.
- **Packet capture (PCAP):** recording of network traffic around the failure time (opened with Wireshark).
- **Control plane trace logs:** time-stamped events from routing protocols and internal state machines.
- **Configuration and "show tech":** the device's full setup, software version, counters and status in one bundle.
- **System logs (syslog):** general messages such as warnings, errors and reboot reasons.
- A standard checklist per failure type was used, so the right data was requested the first time.

6. Lab Reproduction Environment (Test Bed)

What it is (simple words)

A copy of the customer's setup built inside the company lab, where the problem can be triggered on purpose.

Everyday analogy

A flight simulator: you can crash the plane safely as many times as you want to learn why.

What it does in this project

- Lets engineers run debug tools that are too risky or too slow for a customer's live network.
- Proves the fix works before it is sent to the customer.

Minute details

- Same hardware model (or a virtual version), same software version, similar configuration.
- Traffic generators simulate customer load and protocol neighbours.
- Engineers try to recreate the exact trigger: a specific packet, a timing race, a config change, a link going down.
- If the problem cannot be reproduced, the team goes back to collect more evidence (this loop appears in the workflow diagram).

7. Debug Build and Debug Symbols

What it is (simple words)

A version of the same software that keeps extra information so tools can show real function names and code line numbers.

Everyday analogy

A map with street names, compared to a blank map with only lines.

What it does in this project

- Without symbols, GDB shows only memory addresses like 0x4a3f10. With symbols it shows "route_update() in bgp_rib.c, line 812".

Minute details

- Symbol files must match the exact build the customer is running, otherwise the results are wrong.
- Sometimes a special build with extra logging or memory checking is used in the lab.

8. GDB (GNU Debugger)

What it is (simple words)

A tool that lets you look inside a running or crashed program: where it stopped, what functions it was in, and what values its variables had.

Everyday analogy

An airplane's black-box recorder reader. After a crash you read it to see exactly what the plane was doing at the last second.

What it does in this project

- Main tool for **crashes and hangs**.
- Opens the core dump and shows the exact line of code where the crash happened, and why.

Minute details

- **Load the crash:** gdb <program> <core-file> opens the crashed program together with its memory snapshot.
- **bt (backtrace):** shows the chain of function calls at the moment of the crash.
- **frame N / info locals:** jump into one function and see its local variables.
- **print <variable>:** shows a value, for example a pointer that is NULL (empty) when it should not be.
- **info threads / thread apply all bt:** shows every thread; essential for hangs and deadlocks where two threads wait on each other forever.
- **Typical findings:** NULL pointer use, reading past the end of an array, double free, deadlocks, unhandled error cases.
- Can also attach to a live process in the lab and set breakpoints (pause points) to watch the bug happen step by step.

Small example

Backtrace shows a crash in process_update() because a neighbour entry pointer was NULL. The neighbour had been deleted a moment earlier, but the update code still tried to use it.

9. Valgrind

What it is (simple words)

A tool that runs a program under close watch and reports every memory mistake it makes.

Everyday analogy

A strict accountant who checks that every coin borrowed is returned, and that no one spends money that is not theirs.

What it does in this project

- Main tool for **memory leaks and memory corruption**, the bugs that make a device slowly run out of memory or crash randomly.

Minute details

- **Memcheck (default tool):** finds leaks, use of uninitialised memory, reading or writing outside allocated memory, use-after-free, double free.
- **Leak report:** lists memory "definitely lost" with the exact place in code where it was allocated.
- **Massif:** shows how memory use grows over time, to find which feature is eating memory.
- **Helgrind / DRD:** look for thread timing problems (data races).
- It makes programs run much slower (often 10 to 50 times), so it is used in the lab, not on the customer's live device.

Small example

Memory on the router grows by about 2 MB per hour. Valgrind shows each failed session login allocates a buffer that is never freed. After thousands of logins, memory runs out.

10. Wireshark

What it is (simple words)

A tool that opens network traffic recordings and decodes every packet into human-readable fields.

Everyday analogy

Opening every letter in a mailbag and reading who sent it, who it was for, and what it said.

What it does in this project

- Main tool for **traffic loss, wrong packets and protocol disagreements** between devices.
- Proves whether a problem is inside our device, in a neighbour device, or in the network between them.

Minute details

- Opens PCAP files captured on the device, a mirror port, or a network tap.
- **Display filters** narrow millions of packets down to the relevant ones, for example bgp, tcp.analysis.retransmission, or ip.addr == 10.1.1.1.
- **Follow stream** shows one whole conversation in order.
- Spots malformed packets, missing replies, retransmissions, wrong timers, unexpected resets, and MTU (packet size) problems.
- Time stamps are lined up with log time stamps to build one combined timeline.

Small example

Capture shows the neighbour router sent a BGP update with an unusual attribute. Right after that our router reset the session. This points to a parsing bug when handling that attribute.

11. Control Plane Trace Logs

What it is (simple words)

Very detailed, time-stamped diaries that the routing and control software writes about its own decisions and state changes.

Everyday analogy

A ship's captain's log: every course change, every signal received, written down with the time.

What it does in this project

- Main tool for **protocol instability** (sessions going up and down, routes flapping) and timing problems.
- Fills the gap between "what the packets show" and "what the code did".

Minute details

- Records events such as: neighbour up/down, state machine changes (Idle, Connect, Established), route added/removed, timer expired, message received/sent.

- Debug or trace levels can be raised for one protocol to capture more detail, then lowered again.
- Often stored in a circular buffer that overwrites old entries, so it must be collected soon after the failure.
- Reading them in order shows the exact sequence that led to the failure, including race conditions where two events happen in an unexpected order.

Small example

Trace shows: hold timer expired at 02:13:59.8, then neighbour deleted, then at 02:14:00.1 a late update arrives for the deleted neighbour. This matches the GDB crash exactly.

12. Correlation (Joining the Clues)

What it is (simple words)

Putting the findings from every tool on one timeline so they tell a single, consistent story.

Everyday analogy

A detective pinning all photos and notes on a board and connecting them with string.

What it does in this project

- Turns separate observations into one clear explanation of cause and effect.

Minute details

- Align time stamps from core file, logs, captures and syslog (watching out for time zone and clock differences).
- Ask "why?" repeatedly (the 5 Whys method) until reaching a cause that, if fixed, stops the problem.
- Rule out other explanations so the conclusion is proven, not guessed.

13. Root Cause Identification

What it is (simple words)

The final, proven answer to "why did this really happen?"

Everyday analogy

The doctor's diagnosis after the tests come back.

What it does in this project

- Decides what the fix must change.

Minute details

- Types of root cause: code defect, missing error handling, race condition, memory management error, wrong default setting, design gap, or a third-party/neighbour device issue.
- Proof standard: the problem can be triggered on demand in the lab, and stops when the fix is applied.

14. Corrective Action (The Fix)

What it is (simple words)

The change that removes the problem: code patch, hotfix release, or configuration workaround.

Everyday analogy

Replacing the broken car part, plus advice on how to avoid stressing it until the new part arrives.

What it does in this project

- Actually ends the customer's pain, which stops the MTTR clock.

Minute details

- **Immediate workaround:** a safe configuration change the customer can apply today.
- **Permanent fix:** a code change, peer reviewed, built into a hotfix or the next release.
- **Verification:** the lab reproduction is run again with the fix and the problem no longer happens.
- **Regression testing:** automated tests confirm nothing else broke; a new test is added for this bug so it can never silently come back.

15. RCA Report

What it is (simple words)

A written document that explains the problem from start to finish for customers, managers and engineers.

Everyday analogy

The doctor's discharge summary: what you had, what we did, and how to stay healthy.

What it does in this project

- Restores customer trust by showing the problem is understood and controlled.
- Becomes reference material for future similar issues.

Minute details

- **Summary:** what happened, in two or three plain sentences.
- **Impact:** who and what was affected, and for how long.
- **Timeline:** key events with time stamps.
- **Root cause:** the proven reason, with supporting evidence.
- **Corrective action:** the fix and workaround, with versions.
- **Preventive action:** what changes so this kind of bug is caught earlier (new tests, code review rules, monitoring).

16. Knowledge Base and MTTR Metrics

What it is (simple words)

The shared library of past cases and the measurement of how fast issues are resolved.

Everyday analogy

A team's playbook plus a scoreboard.

What it does in this project

- Knowledge base lets the next engineer match a new crash signature to a known bug in minutes.
- MTTR tracking proved the 50% improvement and showed where time was still being lost.

Minute details

- MTTR = total time to resolve all issues / number of issues.
- Example: 30 issues taking 300 days in total gives an MTTR of 10 days. Cutting the total to 150 days gives an MTTR of 5 days, which is 50% faster.

4. Workflow Explained (End to End)

This is the journey of a single customer issue from the moment it is reported to the moment it is closed. The diagram below shows the ten steps and the two "go back" loops.

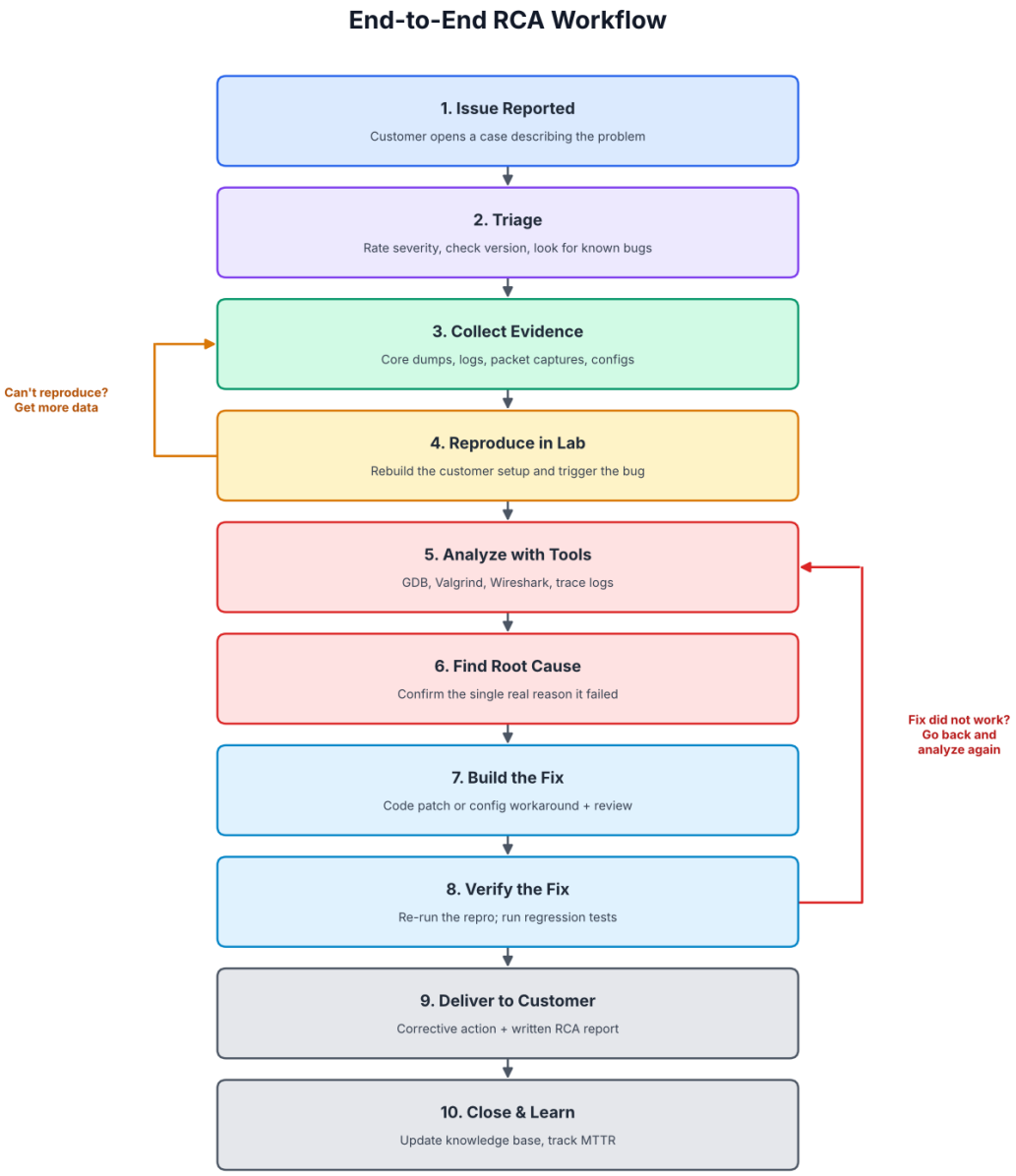


Figure 1: End-to-end RCA workflow

Step 1: Issue Reported

The customer notices something wrong and opens a support case.

- Customer describes the symptom, time, device and impact.

- Case is created in the ticketing system; the MTTR clock starts.
- First-line support checks basics and escalates to engineering if needed.

Output: an open case with a clear problem statement.

Step 2: Triage

Quickly decide how serious it is and what type of problem it is.

- Set severity and priority based on business impact.
- Check the software version against known bugs and release notes.
- Search for matching crash signatures or error messages in past cases.
- Classify the failure type (crash, hang, leak, traffic loss, protocol flap) to pick the first tool.
- Assign an owner.

Output: priority, owner, failure type, and a short investigation plan.

Step 3: Collect Evidence

Get every useful clue in one go.

- Request core dumps, show-tech bundle, syslog, control plane traces and packet captures using a standard checklist.
- If needed, ask the customer to enable extra tracing and capture the next occurrence.
- Verify files are complete, from the right device, and cover the failure time.
- Match the exact software build and get the matching debug symbols.

Output: a complete evidence package attached to the case.

Step 4: Reproduce in Lab

Make the problem happen again in a safe place.

- Build a lab setup that matches the customer's topology, version and configuration.
- Replay traffic or recreate the trigger event.
- If it cannot be reproduced, return to Step 3 for more data (left loop in the diagram).
- Some issues are analysed from customer evidence alone when reproduction is impossible.

Output: a reliable way to trigger the bug on demand.

Step 5: Analyze with Tools

Use the right tool for each type of clue.

- **GDB** on the core dump: backtrace, variables, threads.

- **Valgrind** on the lab build: leaks and memory corruption.
- **Wireshark** on the PCAP: decode and filter the relevant packets.
- **Trace logs**: rebuild the sequence of control plane events.
- Record each finding with its time stamp.

Output: a set of technical findings from each tool.

Step 6: Find the Root Cause

Join the findings into one proven explanation.

- Put all events on one timeline.
- Apply the 5 Whys to go from symptom to true cause.
- Rule out other possibilities.
- Confirm by triggering the bug deliberately in the lab.

Output: a clearly stated, evidence-backed root cause.

Step 7: Build the Fix

Create the corrective action.

- Give the customer a safe workaround if one exists, to stop the pain now.
- Write the code fix; get it peer reviewed.
- Add a unit or regression test that would have caught this bug.

Output: workaround plus a code fix ready for testing.

Step 8: Verify the Fix

Prove the fix works and breaks nothing else.

- Re-run the lab reproduction with the fix: the problem must not happen.
- Run regression and stress tests.
- If the fix fails, go back to Step 5 and analyse again (right loop in the diagram).

Output: a verified fix.

Step 9: Deliver to Customer

Hand over the solution and the explanation.

- Ship a hotfix or point to the fixed release.
- Deliver the RCA report.
- Confirm with the customer that the issue no longer happens in their network.

Output: customer running the fix, RCA report delivered.

Step 10: Close and Learn

Make the next case faster.

- Close the case; the MTTR clock stops.
- Add the crash signature, findings and fix to the knowledge base.
- Share lessons with the team; update checklists and playbooks.
- Track MTTR to measure improvement.

Output: closed case and reusable knowledge.

Which Tool for Which Problem

Problem the customer sees	First tool	What the tool reveals
Device or process crashed / rebooted	GDB	Exact crashing line, call chain and bad variable values
Device frozen or process not responding	GDB (all threads)	Which threads are stuck and what they are waiting for
Memory slowly growing, then failure	Valgrind	Exact code that allocates memory but never frees it
Random crashes in different places	Valgrind	Hidden memory corruption, use-after-free, buffer overflow
Traffic dropped or not reaching destination	Wireshark	Missing, malformed or wrongly addressed packets
Neighbour sessions going up and down	Trace logs + Wireshark	Order of protocol events and the message that triggered the reset
Routes flapping or wrong routes	Trace logs	When and why routes were added or withdrawn

5. Architecture Diagram Explained (End to End)

The architecture diagram shows the whole system as seven layers, from the customer at the top to delivery and learning at the bottom. Data flows downward: a problem enters at the top and leaves as a fix and a report at the bottom.

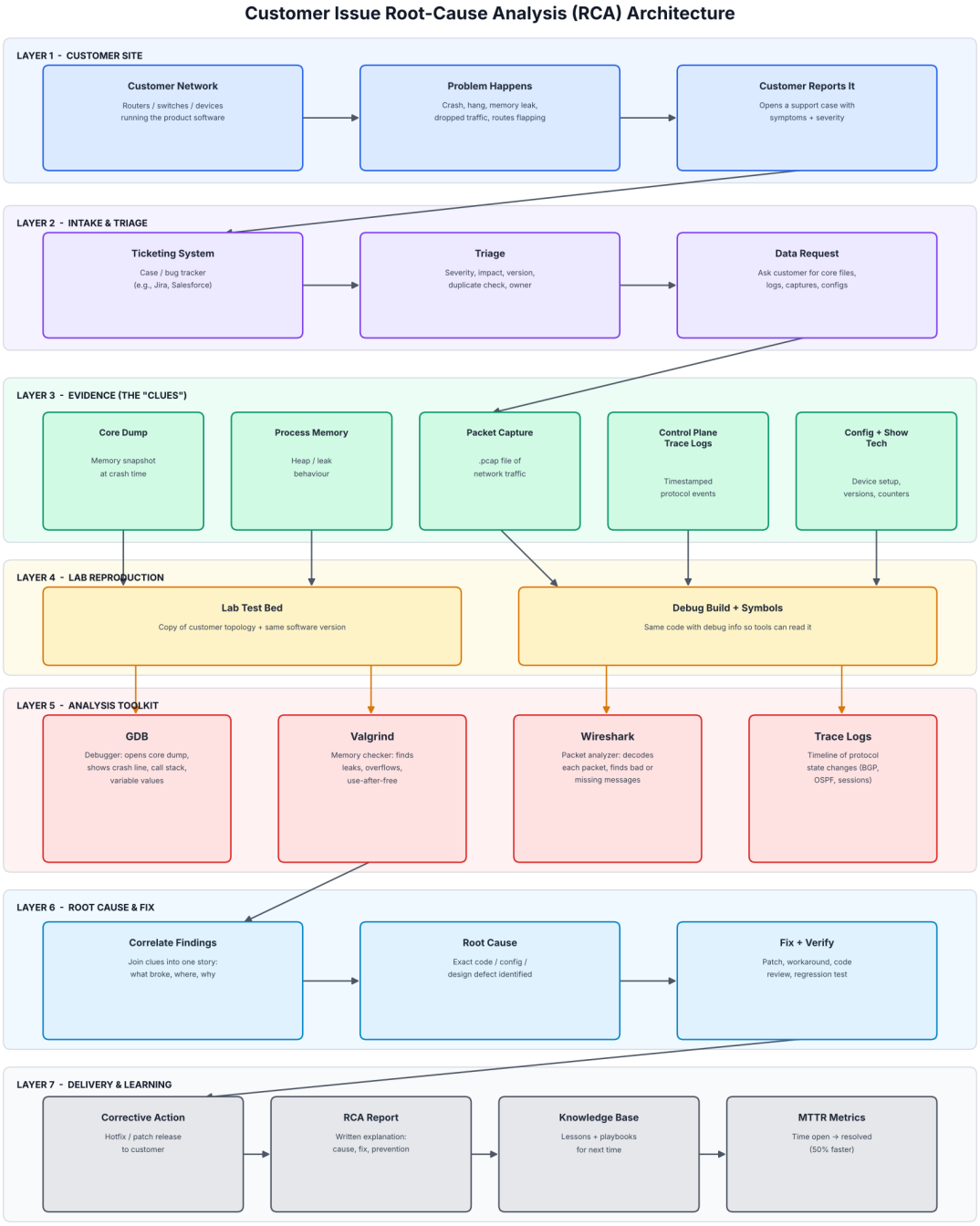


Figure 2: RCA architecture (seven layers)

Layer 1: Customer Site (blue)

Where the problem starts.

- **Customer Network:** the customer's routers and switches running the product software.
- **Problem Happens:** the failure itself, such as a crash, hang, memory leak, dropped traffic or routes flapping.
- **Customer Reports It:** the customer opens a support case with symptoms and severity.
- **Arrows:** left to right show cause and effect: network runs, problem occurs, customer reports. A long arrow then carries the report down into Layer 2.

Layer 2: Intake and Triage (purple)

Where the issue is received and organised.

- **Ticketing System:** stores the case and links it to an engineering bug; time stamps here measure MTTR.
- **Triage:** sets severity, checks version, checks for duplicates, assigns an owner.
- **Data Request:** asks the customer for exactly the evidence needed for this failure type.
- **Arrows:** left to right show the intake order. The arrow from Data Request leads down into the evidence layer.

Layer 3: Evidence, the "Clues" (green)

The five main types of data collected.

- **Core Dump:** memory snapshot at crash time.
- **Process Memory:** heap and leak behaviour data.
- **Packet Capture:** .pcap recording of network traffic.
- **Control Plane Trace Logs:** time-stamped protocol events.
- **Config + Show Tech:** device setup, versions and counters.
- **Arrows:** each piece of evidence flows down into the lab layer. Customer data (crash and memory) feeds the Lab Test Bed; traffic, traces and config feed the Debug Build side so they can be decoded with the right symbols and settings.

Layer 4: Lab Reproduction (yellow)

The safe place to recreate and study the bug.

- **Lab Test Bed:** a copy of the customer topology running the same software version.
- **Debug Build + Symbols:** the same code with debug information so the tools can show real function names and line numbers.

- **Orange arrows:** the lab feeds every analysis tool below it. The Test Bed feeds GDB and Valgrind; the Debug Build feeds Wireshark decoding and trace log interpretation.

Layer 5: Analysis Toolkit (red)

The four investigation tools named in the resume bullet.

- **GDB:** opens the core dump; shows crash line, call stack and variable values.
- **Valgrind:** finds leaks, overflows and use-after-free.
- **Wireshark:** decodes each packet; finds bad or missing messages.
- **Trace Logs:** timeline of protocol state changes (BGP, OSPF, sessions).
- **Arrow:** all tool findings flow down into Correlate Findings in Layer 6.

Layer 6: Root Cause and Fix (light blue)

Where the evidence becomes an answer and a solution.

- **Correlate Findings:** joins all clues into one story: what broke, where and why.
- **Root Cause:** the exact code, configuration or design defect, proven in the lab.
- **Fix + Verify:** patch or workaround, code review, and regression tests.
- **Arrows:** left to right; then a long arrow carries the verified fix down to delivery.

Layer 7: Delivery and Learning (grey)

Where the customer gets the result and the team gets smarter.

- **Corrective Action:** hotfix or patch release given to the customer.
- **RCA Report:** written explanation of cause, fix and prevention.
- **Knowledge Base:** lessons and playbooks saved for next time.
- **MTTR Metrics:** time from open to resolved, showing the 50% improvement.
- **Arrows:** left to right show that each delivered fix produces a report, which feeds knowledge, which is measured in MTTR.

How to Read the Colours and Arrows

Visual element	Meaning
Coloured band (lane)	One layer or stage of the system. Each layer has one colour.
Rounded box	One component inside that layer.
Dark grey arrow	Flow of the case, data or result from one component to the next.

Visual element	Meaning
Orange arrow	The lab environment feeding a specific analysis tool.
Top to bottom	The overall direction: problem in at the top, fix and knowledge out at the bottom.

6. A Full Example Case, Start to Finish

This example shows how all the components and steps work together on one realistic issue.

Stage	What happened
Report	Customer: core router rebooted three times in one week during busy hours. Severity 2.
Triage	Software version had no matching known bug. Reboot reason showed "routing process crashed". Classified as a crash; GDB first.
Evidence	Collected the core dump, show-tech, syslog and BGP trace logs. Asked for a packet capture on the BGP neighbour link for the next event.
GDB	Backtrace showed the crash in the update-handling function, using a neighbour pointer that was already freed.
Trace logs	Neighbour hold timer expired, neighbour deleted, then 300 ms later a late update for that neighbour was processed.
Wireshark	Capture confirmed the neighbour sent a burst of updates right as the link became congested, delaying them.
Valgrind (lab)	Running the lab repro under Valgrind reported "invalid read" (use-after-free) at the exact same line.
Root cause	Race condition: queued updates were not cleared when a neighbour was deleted, so the code used freed memory.
Fix	Workaround: increase hold timer to reduce early deletions. Code fix: flush queued updates on neighbour delete and add a validity check.
Verify	Lab repro ran 24 hours with no crash; regression suite passed; new test added for this race.
Deliver	Hotfix shipped; RCA report sent; customer confirmed stability after two weeks.
Learn	Crash signature added to the knowledge base; code review checklist updated for "delete must clear queues".

7. How the 50% Faster Resolution Was Achieved

Speed did not come from working longer hours. It came from removing waiting time and guesswork at each step.

Improvement	Why it saved time
Standard evidence checklists per failure type	The right data was requested the first time, avoiding several days of back-and-forth with the customer.
Tool-first triage (failure type decides the tool)	No time wasted trying the wrong tool; analysis started immediately.
Ready-to-use lab templates	Common customer topologies could be rebuilt in hours instead of days.
Matching debug symbols archived for every release	GDB analysis could begin the same day the core file arrived.
Correlating all tools on one timeline	Root cause was proven faster, with fewer wrong guesses and rework.
Workaround first, permanent fix second	The customer's pain stopped early while the full fix was built properly.
Knowledge base of crash signatures	Repeat or similar issues were matched to known bugs within minutes.
Clear RCA report template	Reports were written quickly and consistently, so cases closed sooner.

MTTR in numbers (illustrative)

Before: 30 issues x 10 days average = 300 total days.

After: 30 issues x 5 days average = 150 total days.

Result: mean time to resolution cut by 50%.

8. Quick Recap and Interview Talking Points

The project in five lines

1. Customers reported serious problems (crashes, hangs, leaks, traffic loss, protocol flaps) in network software.
2. I led the investigation: triaged, collected the right evidence and reproduced issues in the lab.
3. I used GDB for crashes, Valgrind for memory bugs, Wireshark for traffic and trace logs for protocol timelines.
4. I correlated the findings into proven root causes, delivered fixes and workarounds, and wrote RCA reports.
5. Across 30+ issues, the average time to resolve dropped by about half.

Likely interview questions and short answers

Q: What is the difference between a symptom and a root cause?
A: The symptom is what the customer sees (a reboot). The root cause is the deepest reason (a use-after-free in update handling). Fixing the symptom hides it; fixing the root cause ends it.

Q: How do you analyse a core dump?
A: Load it in GDB with the matching binary and symbols, run a backtrace, inspect frames and variables, check all threads, then confirm the theory in the lab.

Q: When would you use Valgrind instead of GDB?
A: When the problem is about memory over time (leaks) or hidden corruption that crashes in random places. GDB shows where it died; Valgrind shows who damaged the memory.

Q: How did Wireshark help?
A: It showed exactly which packets were exchanged and when, proving whether the trigger came from the network or from inside our device.

Q: What goes into a good RCA report?
A: Summary, impact, timeline, root cause with evidence, corrective action, and preventive action.

Q: How did you cut MTTR by 50%?
A: Standard evidence checklists, tool-first triage, lab templates, a symbol archive, workaround-first delivery and a searchable knowledge base.